MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

Module format Parquet

Apprendre à utiliser des fichiers au format Parquet avec R

MINISTÈRE
DE L'AGRICULTURE
DE LA SOUVERAINETÉ
ALIMENTAIRE ET DE LA FORÊT
Liberté
Égalité
Fraternité

Sommaire

1 C'est quoi un fichier parquet ?

2 Comment utiliser/interroger un fichier parquet ?

3 Écrire des fichiers parquet

4 Manipuler des fichiers parquets avec duckdb

5 Pour en savoir plus

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

0.1 Avant-propos

Ce support ne couvre pas tous les aspects des traitements qu'il est possible de réaliser avec les fichiers au format Parquet mais il constitue une base sur laquelle s'appuyer si vous rencontrez des fichiers Parquet sur Cerise ou ailleurs.

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

1 C'est quoi un fichier parquet ?

MINISTÈRE
DE L’AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

1.1 C’est quoi le format Parquet ?

- Un nouveau format de données …
  - … qui compresse efficacement les fichiers
  - … interopérable
  - conçu pour que les données soient chargées rapidement en mémoire

MINISTÈRE
DE L’AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

1.2 Caractéristiques du format Parquet

- Un stockage au format binaire (pas lisible par un humain)
- Un stockage orienté colonne (par opposition aux csv qui sont orientés lignes)
- Un fichier Parquet contient à la fois les données et des métadonnées

```
read_parquet("data/output.parquet", as_data_frame = FALSE)$schema

# > Table
# 3 rows x 4 columns
# $colonne1 <double not null>
# $colonne2 <string not null>
# $colonne3 <bool not null>
# $colonne4 <date32[day] not null>

read_parquet("data/output.parquet", as_data_frame = FALSE)$schema$metadata

# $auteur
# [1] "DEMESIS/BQIS"

# $description
# [1] "Table test de formation"

# $date creation
```

MINISTÈRE
DE L’AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

1.3 Avantages du format Parquet

- **Des fichiers moins volumineux qu’en csv** 500 Mo en Parquet vs 5 Go en csv
- **Des requêtes plus rapides et efficaces**
  Seulement les données nécessaires sont lues, pas tout le fichier
- Des données conformes à la mise à disposition par le producteur (par exemple, plus de problème de codes communes…)
  => Un format **très efficace pour l’analyse de données** mais **peu adapté à l’ajout de données en continu** ou **à la modification fréquente de données existantes**.

MINISTÈRE
DE L’AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

1.4 Un format qui se démocratise

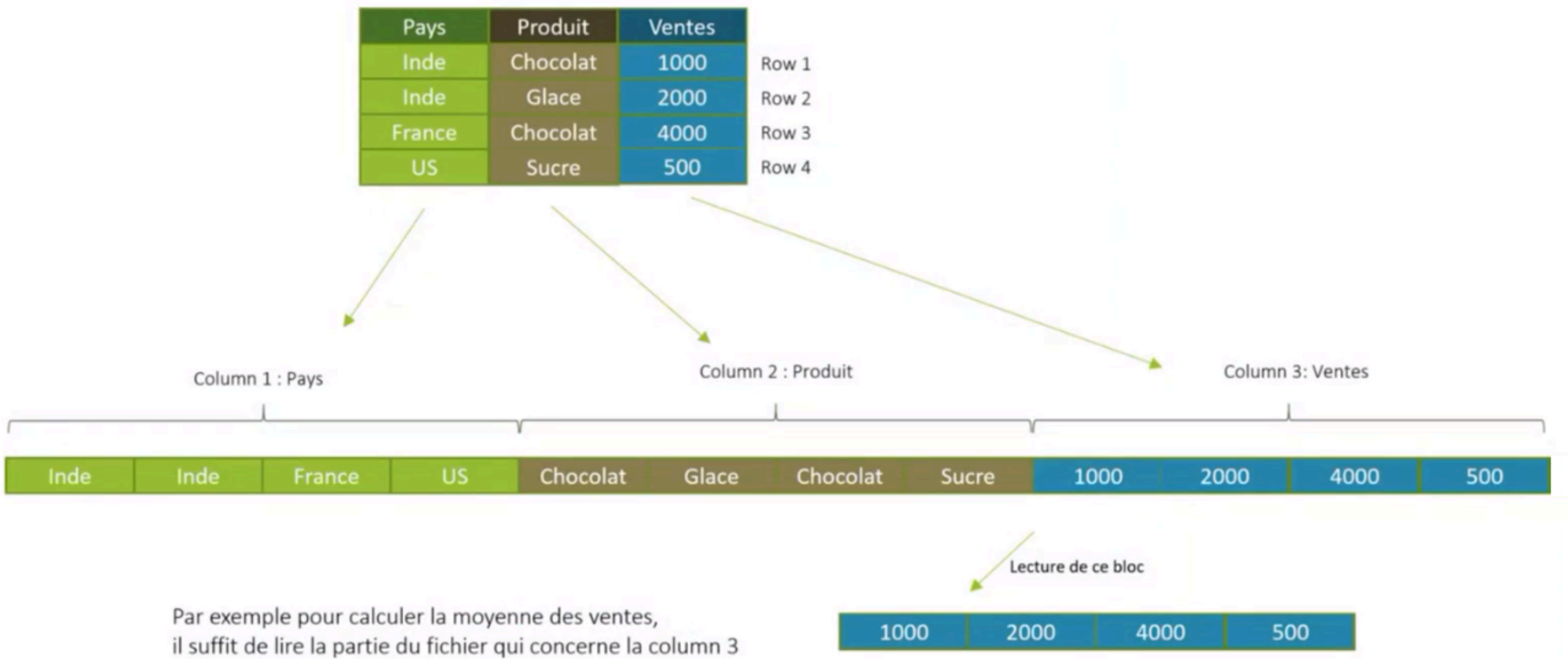
- L’Insee diffuse des données du **recensement de la population** au format Parquet
- Voir **le guide d’utilisation joint** pour manipuler ces données
- Premières diffusions sur **data.gouv** avec les bureaux de vote, les demandes de valeurs foncières, indicateurs pénaux…)
- Prévisualisations des fichiers Parquet possibles avec le nouvel explorateur de données du SSP Cloud ou avec avec l’outil **ParquetViewer**.

MINISTÈRE
DE L’AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

1.5 Constitution d’un fichier parquet (1/3)

Un stockage en mode colonne :



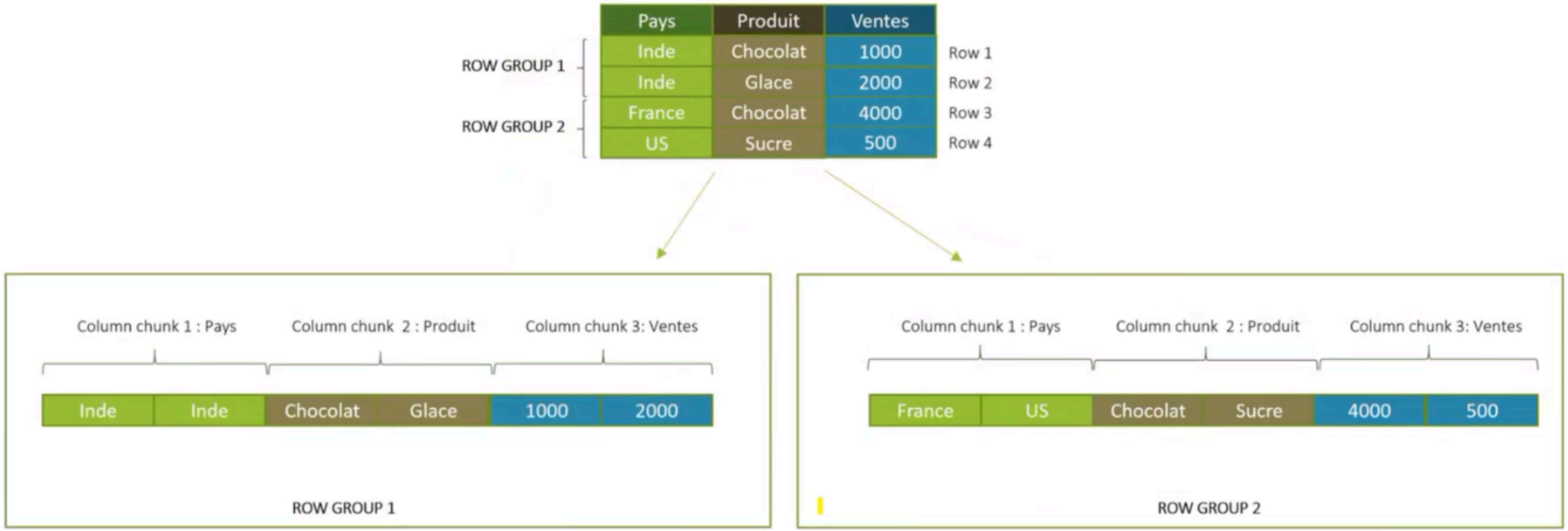
Par exemple pour calculer la moyenne des ventes,
il suffit de lire la partie du fichier qui concerne la column 3

MINISTÈRE
DE L’AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

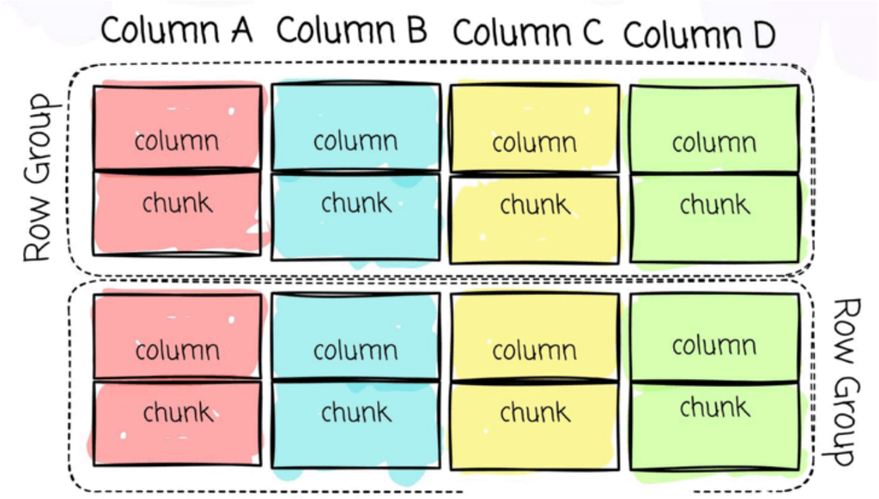
1.6 Constitution d’un fichier parquet (2/3)

Pour les gros fichiers, Parquet dispose les données dans des Row Groups.



MINISTÈRE
DE L’AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

1.7 Constitution d’un fichier parquet (3/3)



*Source : voir ici

Un fichier Parquet est organisé en `row groups` (groupes de lignes de 128 Mo par défaut) :

Chaque `row group` contient l’ensemble des colonnes du jeu de données.

Les données de chaque colonne sont stockées de manière colonnaire au sein du row group :

Chaque colonne est subdivisée en chunks (morceaux) stockés séquentiellement.

Cette structuration permet :

- Une lecture sélective efficace (on peut charger uniquement les colonnes nécessaires).

- Une compression optimisée, car les données similaires sont regroupées.

- Une excellente performance pour les requêtes analytiques sur de grands volumes.

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

2 Comment utiliser/interroger un fichier parquet ?

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.1 Lire un fichier avec read_parquet()

```
1 library(arrow)    # Le package arrow est nécessaire pour travailler avec des fichiers parquet
2 library(dplyr)    # Pour utiliser dplyr
3 library(tictoc)   # Pour le benchmark
```

Pour l'exemple, nous allons prendre une table des exploitations du RA 2020 d'une centaine de MO qui contient 416 478 lignes et 255 colonnes.

```
1 tic()
2 RA2020 <- arrow::read_parquet("data/RA2020_exploitations.parquet")
3 toc()
4 > 1.14 sec elapsed
```

Le résultat obtenu est un objet directement utilisable dans R (ici un data.frame).

Il est possible de sélectionner les colonnes que l'on souhaite importer dans R directement dans la fonction `read_parquet` :

```
1 tic()
2 RA2020_extrait <- arrow::read_parquet("data/RA2020_exploitations.parquet",
3                                       col_select = c("NOM_DOSSIER","SIEGE_REG","SAU_TOT"))
4 toc()
5 > 0.06 sec elapsed
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

2.2 Comparaison avec la lecture d'un fichier rds

Voyons l'écart avec la lecture d'un fichier rds :

```
tic()
RA2020 <- readRDS("data/RA2020_exploitations.rds")
toc()
> 6.15 sec elapsed
```

=> Le temps nécessaire au chargement de la table est d'environ 6 secondes !
L'écart est significatif rien que sur la lecture (X 6).

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

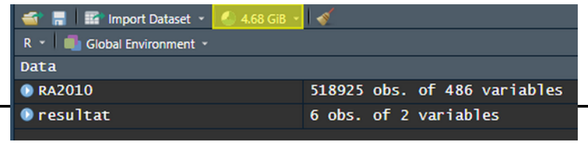
2.3 Des requêtes avec dplyr comme d'habitude

RA2020 est un data.frame : on peut donc utiliser la syntaxe dplyr :

```
resultat <- RA2020 |>
  filter(SIEGE_REG == "93") |>
  group_by(SIEGE_DEP) |>
  summarise(total_SAU = sum(SAU_TOT, na.rm = TRUE))

# A tibble: 6 × 2
  SIEGE_DEP total_SAU
  <chr>         <dbl>
1 04          158946.
2 05           91979.
3 06           41141.
4 13          145713.
5 83           77785.
6 84          112888.
```

- Le temps d'exécution de la requête est d'environ 9 secondes.
- Les ressources consommées sont importantes



MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.4 Lire et exploiter un fichier parquet volumineux

Voici ci-dessous la syntaxe recommandée pour requêter un fichier parquet volumineux :

```
# Établir la connexion aux données
RA2020 <- open_dataset("data/RA2020_exploitations.parquet")  |>
  filter(SIEGE_REG == "93") |>
  group_by(SIEGE_DEP) |>
  summarise(total_SAU = sum(SAU_TOT, na.rm = TRUE)) |>
  collect()
```

=> Avec cette syntaxe, la requête va automatiquement utiliser les variables du fichier Parquet dont elle a besoin (en l'occurrence SIEGE_REG, SIEGE_DEP et SAU_TOT) et minimiser l'occupation de la mémoire vive.

Revenons dans le détail sur cette syntaxe…

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.5 La fonction `open_dataset()` (1/4)

Comme la fonction `read_parquet()`, la fonction `open_dataset()` permet de lire des données stockées en format Parquet.

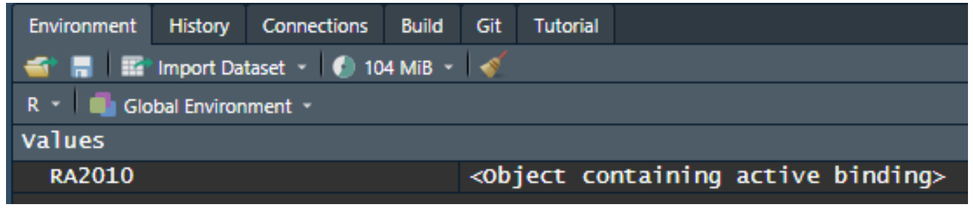
Le résultat obtenu avec la fonction `open_dataset()` n'est plus un **data.frame** mais un **Arrow Table** qui est une structure de données spécifique.

```
1 RA2020 <- open_dataset("data/RA2020_exploitations.parquet")
2
3 class(RA2020)
4
5 > [1] "FileSystemDataset" "Dataset" "ArrowObject" "R6"
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.6 La fonction `open_dataset()` (2/4)

La fonction `open_dataset()` crée un objet qui apparaît dans Values.



L'affichage dans la console d'un Arrow Table affiche uniquement quelques informations.

```
RA2020

> FileSystemDataset with 1 Parquet file
NOM_DOSSIER: string
TYPE_QUESTIONNAIRE: string
SEUIL_IFS: string
CHAMP_GEO: string
COEF_F: double
NUMSTRATE: string
STRATE: string
SIEGENAT: string
SIEGE_CODE_COM: string
SIEGE_LIEUDIT: string
SIEGE_LIEUDIT_CODE_DOM: string
SIEGE_LIB_COM: string
...
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.7 La fonction `open_dataset()` (3/4)

Pour afficher le contenu d'un **Arrow Table**, il faut d'abord le convertir en data.frame avec la fonction `collect()`.

```
RA2020 <- RA2020 |> collect()

class(RA2020)

> [1] "data.frame"

# L'opération ci-dessus est à éviter pour des tables volumineuses, si besoin de visualiser la table, on préfèrera :

extrait_RA2020 <- RA2020 |> slice_head(n = 100) |> collect()
```

Toutefois **rien ne presse** car la grande différence entre manipuler un data.frame et un Arrow Table tient **au moteur d'exécution** :

- Si on manipule un data.frame avec la syntaxe de dplyr, alors c'est le **moteur d'exécution de dplyr** qui fait les calculs 🚲
- Si on manipule un Arrow Table avec la syntaxe de dplyr, alors c'est le **moteur d'exécution d'arrow** (nommé **acero**) qui fait les calculs. Et le moteur d'exécution d'arrow est beaucoup plus efficace et rapide ✈

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

2.8 La fonction `open_dataset()` (4/4)

Il est recommandé de privilégier la fonction `open_dataset()` à la fonction `read_parquet()` pour au moins 2 raisons :

- `open_dataset()` crée une connexion au fichier Parquet mais elle n'importe pas les données contenues dans ce fichier => **une consommation de RAM moins importante !**
- `open_dataset()` peut se connecter à un fichier Parquet unique mais aussi à des **fichiers Parquets partitionnés** (voir plus loin)

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.9 Consulter les métadonnées d'un fichier Parquet (1/4)

Pour obtenir des informations générales sur le fichier (par exemple titre, auteur, date…), il faut utiliser la fonction `read_parquet()` avec l'argument `as_data_frame = FALSE` pour pouvoir accéder aux métadonnées globales via `$schema$metadata`.

```r
# Lire le fichier Parquet en tant qu'Arrow Table avec read_parquet()
arrow_table <- arrow::read_parquet("output.parquet", as_data_frame = FALSE)

# Accéder aux métadonnées du fichier
arrow_table$schema$metadata
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.10 Consulter les métadonnées d'un fichier Parquet (2/4)

Le package {nanoparquet} permet aussi d'obtenir d'autres infos facilement depuis des fichiers parquet :

```
library(nanoparquet)

read_parquet_info("data/RA2020_exploitations.parquet")

# > # A data frame: 1 × 7
  file_name                       num_cols num_rows num_row_groups file_size parquet_version created_by
  <chr>                              <int>    <dbl>          <int>     <dbl>           <int> <chr>
1 data/RA2020_exploitations.parquet      255   416478              1  39896331               2 parquet-cpp-arrow version 9.0.0
```

Le package {parquetize} contient aussi des fonctions similaires :

```
library(parquetize)

get_parquet_info("data/RA2020_exploitations.parquet")

# > # A tibble: 1 × 5
  path                            num_rows num_row_groups num_columns mean_row_group_size
  <chr>                              <int>          <int>       <int>               <dbl>
1 data/RA2020_exploitations.parquet   416478              1         255              416478
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.11 Consulter les métadonnées d'un fichier Parquet (3/4)

- Les types des colonnes :

```
nanoparquet::read_parquet_schema("data/output.parquet")

# A data frame: 4 × 6
# file_name    name     type       r_type    repetition_type logical_type
# * <chr>        <chr>    <chr>      <chr>     <chr>           <I<list>>
#   1 test.parquet colonne1 DOUBLE     double    REQUIRED        <NULL>
#   2 test.parquet colonne2 BYTE_ARRAY character REQUIRED        <STRING>
#   3 test.parquet colonne3 BOOLEAN    logical   REQUIRED        <NULL>
#   4 test.parquet colonne4 INT32      Date      REQUIRED        <DATE>
```

Attention, Parquet propose **2 niveaux de type** :

- Le **bas niveau**
- Le **logical type**

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.12 Consulter les métadonnées d'un fichier Parquet (4/4)

La fonction `read_parquet_schema()` retourne les types de **bas niveau** dans la colonne `type` et les **logical_type** dans la colonne `logical_type`.

- Les types `bas niveau` : c'est la manière dont les données sont stockées physiquement dans le fichier.
- Les `logical types` : c'est une interprétation sémantique des données stockées (ils donnent du sens aux types bas niveau).

Pour aller plus loin sur les métadonnées d'un fichier :

```
1 parquet_metadata("data/RA2020_exploitations.parquet")
```

=> *On obtient des informations sur le schéma, les row_groups, les column chunks…*

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.13 L'évaluation/exécution différée (1/4)

Cela signifie qu'**arrow** se contente de mémoriser les instructions, sans faire aucun calcul tant que l'utilisateur ne le demande pas explicitement.

Il existe **2 fonctions pour déclencher l'évaluation d'un traitement arrow** mais qui présente des différences :

- `collect()` qui renvoie le résultat du traitement sous la forme d'un **data.frame/tibble**
- `compute()` qui renvoie le résultat du traitement sous la forme d'un **Arrow Table**.

La grande différence entre manipuler un tibble et manipuler un Arrow Table tient au moteur d'exécution **:**

- Avec un **data.frame/tibble** => moteur d'exécution de dplyr
- Avec un **Arrow Table** => moteur d'exécution d'arrow (nommé acero) plus efficace que celui de dplyr

Dans les traitements intermédiaires, on privilégiera la fonction `compute()` pour pouvoir utiliser le plus possible ~~le moteur acero.~~

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.14 L'évaluation/exécution différée (2/4)

```
SAU_DEP <- RA2020 |>
  group_by(SIEGE_DEP) |>
  summarise(total_SAU = sum(SAU_TOT, na.rm = TRUE))
class(SAU_DEP)
> [1] "arrow_dplyr_query"

resultats <- SAU_DEP |>
  filter(SIEGE_DEP == "13") |>
  collect()
> # A tibble: 1 × 2
  SIEGE_DEP total_SAU
  <chr>         <dbl>
1 13          145713.
```

Dans l'exemple ci-dessus, **la première étape ne réalise aucun calcul par elle-même, car elle ne comprend ni collect() ni compute()**. L'objet SAU_DEP n'est pas une table et ne contient pas de données, il contient simplement une requête (query) décrivant les opérations à mener sur la table du RA.

arrow analyse la requête avant de l'exécuter, et optimise le traitement pour minimiser le travail.

Dans notre exemple, arrow repère que la requête ne porte en fait que sur le département 13, et commence donc par filtrer les données sur le département avant de sommer la SAU les équipements, de façon à ne conserver que le minimum de données nécessaires et à ne réaliser que le minimum de calculs.

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.15 L'évaluation/exécution différée (3/4)

L'évaluation/exécution différée est très puissante mais **présente des limites**.

On serait tentés d'écrire un traitement entier *en mode lazy* (sans aucun `compute()` ni `collect()` dans les étapes intermédiaires) et de faire un unique `compute()` ou `collect()` tout à la fin du traitement afin que toutes les opérations soient optimisées en une seule étape.

Malheureusement, **le moteur acero** a ses limites notamment sur des traitements trop complexes (ce qui génère des plantages de sessions R).

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.16 L'évaluation/exécution différée (4/4)

QUELQUES CONSEILS POUR ÉLABORER LA BONNE STRATÉGIE AVEC L'ÉVALUATION DIFFÉRÉE :

- Décomposer le traitement en plusieurs étapes puis exécuter chaque étape séparément (avec un `compute()`)
- Définir la bonne longueur des étapes intermédiaires en gardant en tête :
  - D'avoir des étapes de traitement qui **ne dépassent pas 40 lignes de code**
  - Que le séquencement des étapes soit **cohérent** avec l'objet du traitement
  - Plus les **données sont volumineuses** OU les **opérations unitaires sont complexes**, plus **les étapes de traitement doivent être courtes/prudentes**

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.17 Quelques manques sur le moteur acero (1/2)

La liste des fonctions du *tidyverse* supportées par acero est disponible sur cette page.

Il y a (encore) quelques grands absents, notamment :

- `pivot_wider()` et `pivot_longer()` n'ont pas d'équivalent avec **acero**.
- **les empilements de plusieurs tables avec une seule fonction** (`bind_rows()` dans dplyr).
  Avec des Arrow Tables, il faut appeler plusieurs fois ces fonctions (en l'occurence `union()`. Par exemple :

```
resultats <- table1 |>
  union(table2) |>
  union(table3) |>
  compute()
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.18 Quelques manques sur le moteur acero (2/2)

- les *window functions* (ajouter à une table des informations issues d'une agrégation par groupe) comme par exemple :

```
res <- RA2020 |>
  group_by(SIEGE_REG) |>
  mutate(total_SAU = sum(SAU_TOT)) |>
  collect()

> Error: window functions not currently supported in Arrow
Call collect() first to pull data into R.
```

Remarque : le code ci-dessus fonctionne par contre en remplaçant le `mutate()` par un `summarise()`.

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.19 Comment contourner le problème d'acero ? (1/3)

Plusieurs solutions existent :

1. Comme suggéré par R, renoncer à manipuler les données sous forme d'Arrow Table avec le moteur acero en passant par un `collect()` et poursuivre le traitement avec le moteur d'exécution de dplyr (avec des performances moins importantes).

2. Étudier le message d'erreur renvoyé par R et chercher à réécrire d'une autre façon le traitement.

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.20 Comment contourner le problème d'acero ? (2/3)

Exemple pour le point 2 issu d'utilitr :

```
resultats <- bpe_ens_2018_arrow |>
  group_by(DEP) |>
  summarise(
    nb_boulangeries  = sum(NB_EQUIP * (TYPEQU == "B203")),
    nb_poissonneries = sum(NB_EQUIP * (TYPEQU == "B206"))
  ) |>
  compute()

> ! NotImplemented: Function 'multiply_checked' has no kernel matching input types (double, bool); pulling data into R
```

L'erreur vient de l'opération sum(NB_EQUIP * (TYPEQU == "B203")) : **arrow** ne parvient pas à faire la multiplication entre NB_EQUIP (un nombre réel) et (TYPEQU == "B203") (un booléen).

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.21 Comment contourner le problème d'acero ? (3/3)

=> La solution est très simple: **il suffit de convertir (TYPEQU == "B203") en nombre entier avec la fonction as.integer() qui est supportée par acero**.

Le code suivant peut alors être entièrement exécuté par acero:

```
resultats <- bpe_ens_2018_arrow |>
  group_by(DEP) |>
  summarise(
    nb_boulangeries  = sum(NB_EQUIP * as.integer(TYPEQU == "B203")),
    nb_poissonneries = sum(NB_EQUIP * as.integer(TYPEQU == "B206"))
  ) |>
  compute()
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.22 En conclusion sur le package arrow

Le package arrow présente 3 avantages majeurs :

- **Performances élevées** : arrow est très efficace et très rapide pour la manipulation de données tabulaires (nettement plus performant que dplyr par exemple)
- **Usage réduit des ressources** : arrow est conçu pour ne charger en mémoire que le minimum de données. Cela permet de réduire considérablement les besoins en mémoire, même lorsque les données sont volumineuses
- **Facilité d'apprentissage grâce aux approches dplyr et SQL**: arrow peut être utilisé avec les verbes de dplyr (select, mutate, etc.) et/ou avec le langage SQL grâce à DuckDB (voir plus loin).

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.23 Exercice 1

Exercice 1 (premiers contacts avec un fichier parquet + rappels sur les fonctions)

- Ouvrir le fichier parquet situé sous `~/CERISE/03-Espace-de-Diffusion/030_Structures_exploitations/3020_Recensements/RA_2020/01_BASES DIFFUSION RA2020/RA_2020_parquet/RA2020_EXPLOITATIONS_240112.parquet`
- Consulter les types de colonne de ce fichier
- Consulter les 100 premières lignes de ce fichier
- Récupérer dans un vecteur trié les codes régions des lieux principaux de production (SIEGE_REG)
- Récupérer dans un vecteur trié les libellés régions des lieux principaux de production (SIEGE_LIB_REG)
- Ecrire une fonction calculs_RA() qui - pour une région et une table donnée en entrée - conserve uniquement les lignes correspondantes selon la colonne SIEGE_REG, puis groupe la table par SIEGE_DEP et calcule la surface totale SAU (SAU_TOT), la surface totale de céréales (CEREALES_SUR_TOT) et la surface totale d'oléagineux (OLEAG_SUR_TOT) et enfin la part de la surface des céréales dans la SAU totale et la part de la surface des oléagineux dans la SAU totale.
- Utiliser ensuite la fonction calculs_RA() pour calculer ces indicateurs sur l'ensemble des régions présentes dans la table du RA2020 et stocker les résultats dans des fichiers Excel sous votre espace personnel.
  TIPS : pensez à utiliser {purrr} et {openxlsx} par exemple.

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

2.24 Exercice 2

Exercice 2 (`collect()` vs `compute()`)

- Dans votre espace de travail, créer les 3 fichiers parquet suivants :

```r
data_a <- tibble(
  id = rep(1:1000000, each = 10),
  annee = rep(2016:2025, times = 1000000),
  a = sample(letters, 10000000, replace = TRUE)
)

data_b <- tibble(
  id = rep(1:1000000, each = 10),
  annee = rep(2016:2025, times = 1000000),
  b = runif(10000000, 1, 100)
)

data_c <- tibble(
  lettres = sample(letters, 10000000, replace = TRUE),
  classe = sample(c("pommes","poires","melon","fraise"), 10000000, replace = TRUE)
)

write_parquet(data_a, "data_a.parquet")
write_parquet(data_b, "data_b.parquet")
write_parquet(data_c, "data_c.parquet")

rm(data_a)
rm(data_b)
rm(data_c)
gc()
```

- La suite de l'exercice sur la slide suivante…

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

2.25 Exercice 2 (suite)

Exercice 2 (`collect()` vs `compute()`)

a. AVEC `collect()`

Charger les fichiers parquet `data_a` et `data_b` sous forme de data.frame
Créer la table `etape1` en réalisant une jointure à gauche de `data_a` avec `data_b`.
Charger le fichier parquet `data_c` sous forme de data.frame
Filtrer la table `etape1` sur les années supérieures à 2020 puis faire la somme de la colonne `b` selon la colonne `a`
Ajouter la colonne `classe` issue de la table `data_c` dans le tableau final.

b. AVEC `compute()`

Réaliser les mêmes traitements que A) avec des compute() et réduire le temps d'exécution.

3 Écrire des fichiers parquet

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.1 Données peu volumineuses: écrire un seul fichier Parquet (1/2)

En tant que responsable de sources, vous pouvez être amenés à écrire et déposer des fichiers Parquet, par exemple sous Cerise.

Pour cela, on utilise la fonction `write_parquet()`.
Un 1er exemple simple à partir d'un fichier rds:

```r
# Lecture du fichier rds
msa_ns <- readRDS("data/msa_ns_src_2023.rds")

# Écriture des données en format Parquet
write_parquet(x = msa_ns, sink = "data/msa_ns_src_2023.parquet")
```



MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

3.2 Données peu volumineuses: écrire un seul fichier Parquet (2/2)

Un autre exemple un peu plus compliqué à partir de fichier csv contenu dans un zip sur internet :

```r
# Chargement des packages
library(arrow)
library(readr)

# Téléchargement du fichier zip
download.file("https://www.insee.fr/fr/statistiques/fichier/2540004/dpt2021_csv.zip", destfile = "data/dpt2021_csv.zip")
# Décompression du fichier zip
unzip("data/dpt2021_csv.zip", exdir = "data")

# Lecture du fichier CSV
dpt2021 <- read_delim(file = "data/dpt2021.csv")

# Écriture des données en format Parquet
write_parquet(x = dpt2021, sink = "data/dpt2021.parquet"))
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

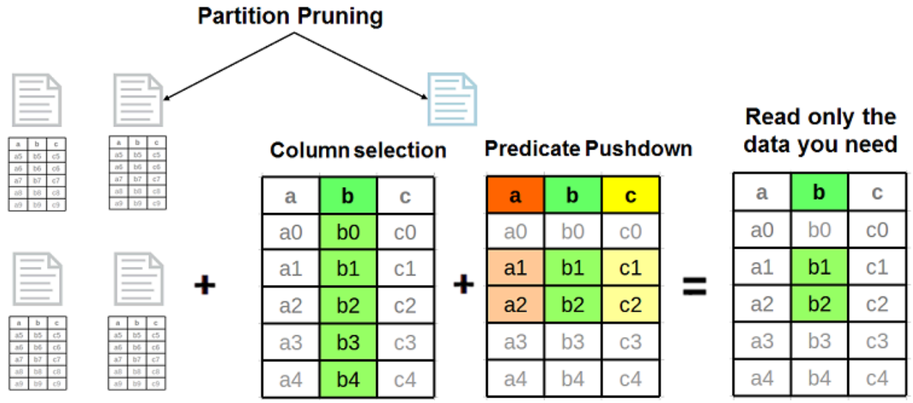
3.3 Données volumineuses: écrire un fichier Parquet partitionné (1/3)

Pourquoi partitionner ?

Par définition, il n'est pas possible de charger seulement quelques lignes d'un fichier Parquet : **on importe nécessairement des colonnes entières**.

Lorsque le fichier Parquet est partitionné, **arrow est capable de filtrer les lignes à importer à l'aide de clés de partitionnement**, ce qui permet d'accélérer l'importation des données.

Le partitionnement permet de travailler sur des fichiers Parquet de plus petite taille et donc de consommer moins de mémoire vive.



MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

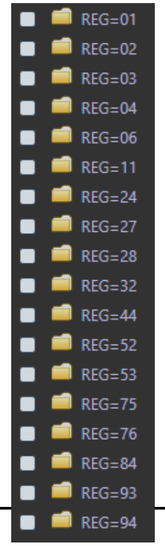
3.4 Données volumineuses: écrire un fichier Parquet partitionné (2/3)

Ça veut dire quoi partitionné ?

Partitionner un fichier revient à le "découper" selon une clé de partionnement (une ou plusieurs variables)

En pratique, l'ensemble des données sera stockée dans plusieurs fichiers au format Parquet.

Voici par exemple comment se présente un fichier Parquet partitionné selon les régions :



MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

3.5 Données volumineuses: écrire un fichier Parquet partitionné (3/3)

Pour écrire des fichiers Parquet partitionnés, on utilise la fonction `write_dataset()`.

Partitionnons notre fichier issu de la MSA **par type d'exploitation et sexe** :

```r
write_dataset(
  dataset = msa_ns,
  path = "data/msa_ns",
  partitioning = c("TYPE_EXP","SEXE"), # les variables de partitionnement
  format = "parquet"
)
```

Voici un aperçu de l'arborescence créée (:

```
data/msa_ns
├── TYPE_EXP=1
│   ├── SEXE=1
│   │   └── part-0.parquet
│   └── SEXE=2
│       └── part-0.parquet
├── TYPE_EXP=2
│   ├── SEXE=1
│   │   └── part-0.parquet
│   └── SEXE=2
│       └── part-0.parquet
└── TYPE_EXP=3
    ├── SEXE=1
    │   └── part-0.parquet
    └── SEXE=2
        └── part-0.parquet
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

3.6 Forcer les types des colonnes lors de l'écriture de fichiers Parquet

En tant que responsable de sources par exemple, vous pouvez forcer le typage des colonnes d'un fichier Parquet.
Pour cela, assurez-vous que les colonnes du data.frame R sont au bon type.
Si ce n'est pas le cas, utilisez les fonctions de conversion `as.character()`, `as.integer()`, `as.Date()` ...

```r
#Créez un DataFrame R en spécifiant le type de chaque colonne
df <- data.frame(
  colonne1 = as.integer(c(1, 2, 3)),
  colonne2 = as.character(c("A", "B", "C")),
  colonne3 = as.logical(c(TRUE, FALSE, TRUE)),
  colonne4 = as.numeric(35.43, 29.93, 17.02))

write_parquet(table, "output.parquet")
```

Une autre façon de forcer le typage des colonnes est d'utiliser un `schema`. Cependant, cela apporte de la complexité au code pas forcément utile.
~~En cas de besoin, un exemple est disponible sur cette page.~~

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

3.7 Ajouter des métadonnées générales sur un fichier Parquet (1/3)

A partir d'un data.frame R et avant l'écriture du fichier Parquet correspondant, il est possible d'ajouter des métadonnées générales sur un fichier Parquet.

Soit le data.frame suivant :

```r
df <- data.frame(
  colonne1 = c(1, 2, 3),
  colonne2 = c("A", "B", "C"),
  colonne3 = c(TRUE, FALSE, TRUE),
  colonne4 = as.Date(c("2025-01-01", "2025-01-02", "2025-01-03")))
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

3.8 Ajouter des métadonnées générales sur un fichier Parquet (2/3)

Avec uniquement le package {arrow}, on peut ajouter des métadonnées comme ceci :

```r
# Conversion en Arrow Table
df_arrow <- arrow_table(df)

# Ajouter des métadonnées
df_arrow <- df_arrow$ReplaceSchemaMetadata(c(
  auteur = "DEMESIS/BQIS",
  description = "Table test de formation",
  date_creation = as.character(Sys.Date())
))

write_parquet(df_arrow, "data/output.parquet")
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

3.9 Ajouter des métadonnées générales sur un fichier Parquet (3/3)

Avec le package {nanoparquet}, le code est plus court.
Attention ici, la fonction `write_parquet()` utilisée est issue du package {nanoparquet} et non {arrow} (qui ne propose pas l'argument `metadata` et dont les noms des autres arguments sont différents).

```
nanoparquet::write_parquet(
  x = df,
  file = "data/output.parquet",
  metadata = c(
    auteur = "DEMESIS/BQIS",
    description = "Table test de formation",
    date_creation = as.character(Sys.Date()))
  )
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.10 Industrialiser la conversion de vos fichiers ?

- Le package R **parquetize** permet de faciliter la conversion de données au format Parquet.
- Plusieurs formats supportés csv, json, rds, fst, SAS, SPSS, Stata, sqlite…
- Propose des **solutions de contournement** pour les fichiers très volumineux.

Un exemple issu de la documentation :

```
Conversion from a local rds file to a partitioned parquet file :: 12
rds_to_parquet(
path_to_file = system.file("extdata","iris.rds",package = "parquetize"),
path_to_parquet = tempfile(fileext = ".parquet"),
partition = "yes",
partitioning = c("Species")
)

#> Reading data...
#> Writing data...
#> ✔ Data are available in parquet dataset under /tmp/RtmptNiaDm/file1897441ca0c0.parquet
#> Writing data...

#> Reading data...
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

3.11 Comment bien utiliser des fichiers partitionnés avec arrow (1/2)

La fonction `open_dataset()` permet **d'ouvrir une connexion vers un fichier Parquet partitionné**.

L'utilisation de la fonction `open_dataset()` est **similaire au cas dans lequel on travaille avec un seul fichier Parquet**.

Il y a toutefois 2 différences :

- Le chemin indiqué n'est pas celui d'un fichier .parquet, mais le chemin d'un répertoire dans lequel se trouve le fichier Parquet partitionné.
  Il est préférable d'indiquer le nom et le type de la ou des variable(s) de partitionnement.

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

3.12 Comment bien utiliser des fichiers partitionnés avec arrow (2/2)

Un exemple avec les données de la MSA :

```r
# Établir la connexion au fichier Parquet partitionné
donnees_msa <- open_dataset(
  "data/msa_ns", # Ici, on met le chemin d'un répertoire
  hive_style = TRUE,
  partitioning = arrow::schema(TYPE_EXP = arrow::utf8(), SEXE = arrow::utf8()) # Les variables de   partitionnement
)

# Définir la requête
resultats_msa <- donnees_msa |>
  filter(TYPE_EXP == "2" & SEXE == "1") |> # Ici, on filtre selon les clés de partitionnement
  select(DEPT, RC_CHEF) |>
  collect()
```

Ce qui donne :

```r
resultats_msa

> resultats_msa
# A tibble: 62,195 × 2
   DEPT  RC_CHEF
 * <chr> <chr>
 1 11    1461
 2 11    2910
 3 11    1528
 4 11    4493
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

3.13 Conseils lors de l'utilisation de fichiers partitionnés

Afin de tirer au mieux profit du partitionnement, il est conseillé de **filtrer les données de préférence selon les variables de partitionnement** (dans notre exemple, `TYP_EXP` et `SEXE`).

Il est fortement recommandé de **spécifier le type des variables de partitionnement avec l'argument partitioning**.

Cela évite des erreurs typiques: le code du département est interprété à tort comme un nombre et aboutit à une erreur à cause de la Corse…

L'argument `partitioning` s'utilise en construisant un schéma qui précise le type de chacune des variables de partitionnement.

Voir **cette page** pour la liste des types supportés.

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

3.14 Dernier conseil avec `arrow`

Il est recommandé de définir les deux options suivantes au début de votre script.

Cela autorise arrow à utiliser plusieurs processeurs à la fois, ce qui accélère les traitements :

```r
Autoriser arrow à utiliser plusieurs processeurs en même temps
options(arrow.use_threads = TRUE)

# Définir le nombre de processeurs utilisés par arrow
# 10 processeurs sont suffisants dans la plupart des cas
arrow:::set_cpu_count(parallel::detectCores() %/% 2)
```

4 Manipuler des fichiers parquets avec duckdb

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.1 Qu'est-ce que duckdb ?

DuckDB est un projet open-source qui propose un moteur SQL optimisé pour **réaliser des travaux d'analyse statistique sur des bases de données**.

Plusieurs avantages :

- Un **moteur portable** utilisable avec **plusieurs langages** (R, Python, Javascript…) et **plusieurs OS** (Windows, Linux, MacOS…)
- Une **installation** et une **utilisation** très **facile**
- Un **moteur SQL** capable d'utiliser des données au format Parquet **sans les charger complètement en mémoire**.

> **Note**
>
> Il faut bien distinguer le projet DuckDB du package R duckdb qui propose simplement une façon d'utiliser Duckdb avec R.

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.2 À quoi sert le package duckdb + installation ?

Du point de vue d'un statisticien utilisant R, le package duckdb permet de faire trois choses :

- Importer des données (exemples: fichiers CSV, fichiers Parquet)
- Manipuler des données avec la syntaxe dplyr, ou avec le langage SQL
- Écrire des données au format Parquet.

Installation de duckdb

Il suffit d'installer le package duckdb, qui contient à la fois DuckDB et une interface pour que R puisse s'y connecter.

Bonne nouvelle sur la version de Cerise mis à disposition en 2025, le package duckdb sera installé par défaut ! 🎉

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.3 Connexions avec duckdb

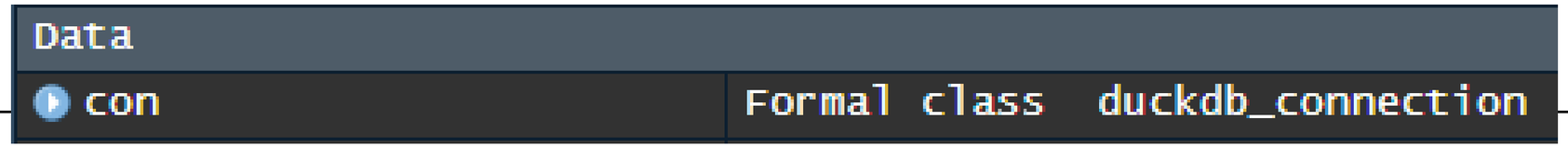
Pour utiliser duckdb, il n'est pas nécessaire de connaître le langage SQL car il est possible d'utiliser duckdb avec la syntaxe dplyr.

duckdb est une base de données distante et s'utilise comme telle : il faut ouvrir une connexion, puis "charger" les données dans la base de données pour les manipuler.

```
1 con <- DBI::dbConnect(drv = duckdb::duckdb())
```

Plusieurs remarques :

- Cette commande crée une nouvelle base de données duckdb **dans la mémoire vive**.
- Cette base de données ne contient aucune donnée lorsqu'elle est créée. L'objet con apparaît dans l'onglet Data de l'environnement RStudio, mais la liste des tables n'y est pas directement accessible



MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.4 Déconnexion avec duckdb

À la fin du traitement ou du programme, on ferme la connexion avec le code ci-dessous. L'option `shutdown` est importante : elle permet de fermer complètement la session duckdb et de libérer la mémoire utilisée.

```
1 DBI::dbDisconnect(con, shutdown = TRUE)
```

Important

Si on n'utilise pas cette option, il arrive souvent que des connexions à moitié ouvertes continuent à consommer des ressources !!!

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.5 Chargement de données issues de la session R

La fonction `duckdb_register()` permet de charger dans duckdb des données présentes en mémoire dans la session R.

Cette méthode a l'avantage de ne pas recopier les données: elle se contente d'établir un lien logique entre la base de données duckdb et un objet de la session R.

L'objet créé dans la base est **une vue** dans le catalogue "temp". La durée d'existence de cette vue est le temps de la connexion.

```
# Création de la vue "RA2020_duckdb"
con |> duckdb::duckdb_register(
  name = "RA2020_duckdb",
  df = RA2020)
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.6 Accéder à une table d'une base DuckDB

Pour vérifier que le chargement des données a bien fonctionné, **la fonction `tbl()` permet d'accéder à une table de la base de données grâce à son nom (entre double quotes).**

```
con |> tbl("RA2020_duckdb")

# Source:   table<RA2020_duckdb> [?? x 255]
# Database: DuckDB v1.1.0 [damien.dotta@Windows 10 x64:R 4.3.0/:memory:]
   NOM_DOSSIER TYPE_QUESTIONNAIRE SEUIL_IFS CHAMP_GEO COEF_F NUMSTRATE  STRATE     SIEGENAT SIEGE_CODE_COM SIEGE_LIEUDIT
   <chr>       <chr>              <chr>     <chr>      <dbl> <chr>      <chr>      <chr>    <chr>          <chr>
 1 AAAAAAA     2                  1         1           1.22 2500000004 25000ERICA 10       14406          NA
 2 BBBBBBB     2                  1         1          10.3  2105161843 2105161843 10       51303          NA
 3 CCCCCCC     3                  1         2           1    0101030406 EXH_GEO    10       97407          CHEMIN DES ANGLAIS
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.7 Chargement de données sur le disque au format Parquet (1/2)

Pour charger des données situés sur Cerise par exemple, la fonction `tbl()` peut aussi être directement utilisée en renseignant le chemin du fichier Parquet.

```
con %>% tbl("read_parquet('data/RA2020_exploitations.parquet')")
# Ou plus succinct (pas besoin de read_parquet() si pas besoin d'y passer des arguments)
con %>% tbl('data/RA2020_exploitations.parquet')

# Source:   SQL [?? x 255]
# Database: DuckDB v1.1.0 [damien.dotta@Windows 10 x64:R 4.3.0/:memory:]
   NOM_DOSSIER TYPE_QUESTIONNAIRE SEUIL_IFS CHAMP_GEO COEF_F NUMSTRATE  STRATE     SIEGENAT SIEGE_CODE_COM SIEGE_LIEUDIT
   <chr>       <chr>              <chr>     <chr>      <dbl> <chr>      <chr>      <chr>    <chr>          <chr>
 1 AAAAAAA     2                  1         1           1.22 2500000004 25000ERICA 10       14406          NA
 2 BBBBBBB     2                  1         1          10.3  2105161843 2105161843 10       51303          NA
 3 CCCCCCC     3                  1         2           1    0101030406 EXH_GEO    10       97407          CHEMIN DES ANGLAIS
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.8 Chargement de données sur le disque au format Parquet (2/2)

Avec des fichiers partitionnés, on peut utiliser la syntaxe suivante.
`**/*.parquet` est un motif qui indique que vous souhaitez lire, dans tous les sous-dossiers quelque soit le niveau (`**`), l'ensemble des fichiers parquets (`*.parquet`) qui s'y trouvent.

```
con %>% tbl('data/msa_ns/**/*.parquet')

# Source:   SQL [?? x 293]
# Database: DuckDB v1.1.0 [damien.dotta@Windows 10 x64:R 4.3.0/:memory:]
   A_IDENT          DEPT  RESID CANTON COMMUNE DOM_FISC SUP   NAF     SMI SITU_FAM NAIS_JO NAIS_MO NAIS_AN AF_MAJ AF_MIN AF_AVA
   <chr>            <chr> <chr> <chr>  <chr>      <dbl> <chr> <chr> <dbl>    <dbl> <chr>   <chr>   <chr>    <dbl>  <dbl>  <dbl>
 1 XXXXXXXXXXXXXX… 11    11    10     288            1 0     NA        0        1 25      05      1969         0      0      0
 2 YYYYYYYYYYYYYY… 11    11    01     281            1 2903  NA        0        1 30      01      2000         0      0      0
 3 ZZZZZZZZZZZZZZ… 11    11    16     269            1 368   NA        0        1 07      01      2000         0      0      0
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.9 Afficher la liste des tables d'une base Duckdb (1/2)

La fonction `dbListTables()` est utile pour afficher les noms des tables présentes dans une base de données.

Une illustration pour mieux comprendre :

```
# On se déconnecte
DBI::dbDisconnect(con, shutdown = TRUE)

# On crée une nouvelle connexion
con <- DBI::dbConnect(drv = duckdb::duckdb())

# Affichage de la liste des tables
dbListTables(con)
# > character(0)

con |> duckdb::duckdb_register(
  name = "iris_duckdb",
  df = iris)

con %>% tbl("read_parquet('data/RA2020_exploitations.parquet')")
```

=> Question : combien de tables va maintenant renvoyer la fonction `dbListTables()` ?

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.10 Afficher la liste des tables d'une base Duckdb (2/2)

Réponse… UNE SEULE !

```r
# Affichage de la liste des tables
dbListTables(con)
# > "iris_duckdb"

# On se déconnecte
DBI::dbDisconnect(con, shutdown = TRUE)
```

En effet, lorsqu'on utilise la fonction `tbl()`, celle-ci ne charge pas les données ni dans la mémoire de R ni dans celle de DuckDB.

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.11 Afficher la liste des colonnes d'une table d'une base Duckdb

La fonction `dbListFields()` est utile pour afficher les noms des colonnes d'une table présente dans une base de données.

```r
# On crée une nouvelle connexion
con <- DBI::dbConnect(drv = duckdb::duckdb())

con |> duckdb::duckdb_register(
  name = "iris_duckdb",
  df = iris)

con |> DBI::dbListFields("iris_duckdb")
# > [1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"

# On se déconnecte
DBI::dbDisconnect(con, shutdown = TRUE)
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.12 Accéder aux logical_types des colonnes d'un fichier parquet avec duckdb

En complément des instructions déjà vues dans la partie 2 de la formation, on peut accéder aux logical_types des colonnes d'un fichier Parquet en utilisant la requête suivante :

```
con <- DBI::dbConnect(drv = duckdb::duckdb())
dbGetQuery(con, "DESCRIBE FROM read_parquet('data/output.parquet')")

# column_name column_type null  key default extra
# 1    colonne1      DOUBLE  YES <NA>    <NA>  <NA>
# 2    colonne2     VARCHAR  YES <NA>    <NA>  <NA>
# 3    colonne3     BOOLEAN  YES <NA>    <NA>  <NA>
# 4    colonne4        DATE  YES <NA>    <NA>  <NA>
```

Remarque : **La fonction dbGetQuery() du package DBI permet de récupérer un data.frame dont le contenu est le résultat d'une requête.**

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.13 Accéder aux métadonnées générales d'un fichier parquet avec duckdb

La fonction `parquet_metadata()` peut également être utilisée dans une requête écrite avec duckdb :

```
1 con <- DBI::dbConnect(drv = duckdb::duckdb())
2 dbGetQuery(con, "SELECT * FROM parquet_metadata('data/fr_immp_transactions.parquet')")
```

row_group_id	row_group_num_rows	row_group_num_colu...	row_group_bytes	column_id	file_off...	num_values	path_in_schema	type	stats_min	stats_max	stats_null_count	stats_distinct_count	stats_min_value	stats_max_value	compression
0	1048576	13	54792718	0	0	1048576	id_transaction	INT32	1	1985836	0	0	1	1985836	SNAPPY
0	1048576	13	54792718	1	0	1048576	date_transaction	INT64	2014-01-02 00:00:00	2024-06-29 00:00:00	0	0	2014-01-02 00:00:00	2024-06-29 00:00:00	SNAPPY
0	1048576	13	54792718	2	0	1048576	prix	DOUBLE	-0.0	641855000.0	0	0	-0.0	641855000.0	SNAPPY
0	1048576	13	54792718	3	0	1048576	departement	BYTE_ARRAY			0	0	01	14	SNAPPY
0	1048576	13	54792718	4	0	1048576	id_ville	INT32	1	834	0	0	1	834	SNAPPY
0	1048576	13	54792718	5	0	1048576	ville	BYTE_ARRAY			0	0	ABBECOURT	YZEURE	SNAPPY
0	1048576	13	54792718	6	0	1048576	code_postal	INT32	1000	48250	0	0	1000	48250	SNAPPY
0	1048576	13	54792718	7	0	1048576	adresse	BYTE_ARRAY			0	0	A CHANVES	9Z RUE GABRIEL VICAI...	SNAPPY
0	1048576	13	54792718	8	0	1048576	type_batiment	BYTE_ARRAY			0	0	Appartement	Maison	SNAPPY
0	1048576	13	54792718	9	0	1048576	n_pieces	INT32	0	90	0	0	0	90	SNAPPY
0	1048576	13	54792718	10	0	1048576	surface_habitable	INT32	0	3640	0	0	0	3640	SNAPPY
0	1048576	13	54792718	11	0	1048576	latitude	DOUBLE	42.5861157352246	50.1638368446025	0	0	42.5861157352246	50.1638368446025	SNAPPY
0	1048576	13	54792718	12	0	1048576	longitude	DOUBLE	-1.14668362067864	7.68079585150826	0	0	-1.14668362067864	7.68079585150826	SNAPPY
1	1048576	13	54577651	0	0	1048576	id_transaction	INT32	1198799	3782564	0	0	1198799	3782564	SNAPPY
1	1048576	13	54577651	1	0	1048576	date_transaction	INT64	2014-01-01 00:00:00	2024-06-29 00:00:00	0	0	2014-01-01 00:00:00	2024-06-29 00:00:00	SNAPPY
1	1048576	13	54577651	2	0	1048576	prix	DOUBLE	-0.0	2086000000.0	0	0	-0.0	2086000000.0	SNAPPY
1	1048576	13	54577651	3	0	1048576	departement	BYTE_ARRAY			0	0	14	29	SNAPPY
1	1048576	13	54577651	4	0	1048576	id_ville	INT32	1	764	0	0	1	764	SNAPPY
1	1048576	13	54577651	5	0	1048576	ville	BYTE_ARRAY			0	0	ABBANS DESSOUS	YVRAC-ET-MALLEYRA...	SNAPPY
1	1048576	13	54577651	6	0	1048576	code_postal	INT32	5700	43450	0	0	5700	43450	SNAPPY
1	1048576	13	54577651	7	0	1048576	adresse	BYTE_ARRAY			0	0	A BERMOTTE	9X RUE DE LA PARCHE...	SNAPPY
1	1048576	13	54577651	8	0	1048576	type_batiment	BYTE_ARRAY			0	0	Appartement	Maison	SNAPPY

=> On retrouve des informations très intéressantes sur le mode de compression utilisé lors de l'écriture du fichier Parquet (SNAPPY), des statistiques sur les row groups (min, max), l'encodage…

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.14 Requêtage avec dplyr

Le package R duckdb a été écrit de façon à pouvoir manipuler les données avec la syntaxe de dplyr, c'est très pratique !

Comme avec le package {arrow} on utilise cette syntaxe avec les fonctions `collect()` ou `compute()`.

```r
# On crée une nouvelle connexion
con <- DBI::dbConnect(drv = duckdb::duckdb())

# Etablissement de la connexion au fichier Parquet
RA2020_dataset <- con %>% tbl('data/RA2020_exploitations.parquet')

# Traitement avec dplyr et un collect()
resultat <- RA2020_dataset |>
  filter(SIEGE_REG == "93") |>
  group_by(SIEGE_DEP) |>
  summarise(total_SAU = sum(SAU_TOT, na.rm = TRUE)) |>
  collect()

# On se déconnecte
DBI::dbDisconnect(con, shutdown = TRUE)
```

```
> resultat
# A tibble: 6 × 2
  SIEGE_DEP total_SAU
  <chr>         <dbl>
1 84          112888.
2 05           91979.
3 04          158946.
4 13          145713.
5 83           77785.
6 06           41141.
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.15 Examen de la requête SQL construite par duckdb

Quand on manipule des objets avec duckdb, on construit des requêtes SQL.
Le package duckdb se contente de traduire le code dplyr en SQL.
La fonction `show_query()` permet de consulter la requête SQL qui a été exécutée par duckdb.

```
# Traitement avec dplyr et un collect()
resultat <- RA2020_dataset |>
  filter(SIEGE_REG == "93") |>
  group_by(SIEGE_DEP) |>
  summarise(total_SAU = sum(SAU_TOT, na.rm = TRUE)) |>
  show_query()

# > <SQL>
SELECT SIEGE_DEP, SUM(SAU_TOT) AS total_SAU
FROM "data/RA2020_exploitations.parquet"
WHERE (SIEGE_REG = '93')
GROUP BY SIEGE_DEP
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.16 Requêtage avec SQL

Si vous avez des connaissances en SQL, il est bien sûr tout à fait possible de requêter une base DuckDB avec ce langage.
La requête est alors passée dans la fonction `DBI::dbGetQuery()` :

```
# On crée une nouvelle connexion
con <- DBI::dbConnect(drv = duckdb::duckdb())

chemin_donnees <- 'C:/Users/damien.dotta/DEMESIS/Formations_R/Formation_R_perfectionnement/formation-R-perf-06-parquet/data'

# Execution de la requete
resultatSQL <- DBI::dbGetQuery(
   con,
   paste0("SELECT SIEGE_DEP, SUM(SAU_TOT) AS total_SAU
    FROM '", file.path(chemin_donnees,'RA2020_exploitations.parquet'),"'
    WHERE (SIEGE_REG = '93')
    GROUP BY SIEGE_DEP"))
# resultatSQL est un data.frame directement utilisable dans R
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.17 Sauvegarder des résultats intermédiaires - dans des fichiers Parquet

Lorsque le traitement est long, vous devez le découper et stocker quelque part vos résulats intermédiaires de manière à ne pas tout recalculer entièrement à chaque fois.

1. Une 1ère solution consiste à écrire ces résultats intermédiaires dans des fichiers Parquet :

```r
# On crée une nouvelle connexion
con <- DBI::dbConnect(drv = duckdb::duckdb())

# Etablissement de la connexion au fichier Parquet
RA2020_dataset <- con %>% tbl('data/RA2020_exploitations.parquet')

# A la fin du traitement, on écrit un fichier parquet intermédiaire
RA2020_dataset |>
  filter(SIEGE_REG == "93") |>
  # Conversion dans un format compatible avec arrow
  arrow::to_arrow() |>
  arrow::write_parquet("data/table_intermediaire.parquet")
```

```r
# Reprise des traitements + tard avec le moteur SQL de duckdb
final <- arrow::open_dataset("data/table_intermediaire.parquet") |>
  # Conversion dans un format compatible avec duckdb
  arrow::to_duckdb(con) |>
  group_by(SIEGE_DEP) |>
  summarise(total_SAU = sum(SAU_TOT, na.rm = TRUE)) |>
  collect()

# On se déconnecte
DBI::dbDisconnect(con, shutdown = TRUE)
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.18 Sauvegarder des résultats intermédiaires - avec une BDD

2. Une 2ème solution consiste à écrire dans une BDD ces résultats intermédiaires :

```
# On crée une nouvelle connexion
# + création d'une BDD vide avec le paramètre dbdir
con <- DBI::dbConnect(drv = duckdb::duckdb(),
                      dbdir = "data/BDD_formation.db")

# Etablissement de la connexion au fichier Parquet
RA2020_dataset <- con %>% tbl('data/RA2020_exploitations.parquet')

# Traitement avec dplyr et un compute()
RA2020_dataset |>
  filter(SIEGE_REG == "93") |>
  compute(name = "resultat_interm",
          temporary = FALSE)

```

```
# Affichage de la liste des tables
dbListTables(con)
# [1] "resultat_interm"

# Reprise des traitements + tard avec le moteur SQL de duckdb
final2 <- con |> tbl("resultat_interm") |>
  group_by(SIEGE_DEP) |>
  summarise(total_SAU = sum(SAU_TOT, na.rm = TRUE)) |>
  collect()

# On se déconnecte
DBI::dbDisconnect(con, shutdown = TRUE)
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.19 A retenir sur les 2 méthodes de sauvegarde des résultats intermédiaires

- La première méthode avec **{arrow} et la sauvegarde dans des fichiers Parquet** est généralement **la plus rapide**.

- La seconde avec **compute() et la sauvegarde dans une BDD** est **la plus efficace en terme d'occupation mémoire**.

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.20 Options à utiliser

Lors d'une connexion à une base, des paramètres de `dbConnect()` peuvent être très utiles lorsqu'on travaille sur un espace comme Cerise où les ressources sont partagées.

Par exemple :

```
con_dbnew <- dbConnect(
  drv = duckdb::duckdb(),
  dbdir=db_file,
  config=list("memory_limit"="10GB", # On définit un plafond de 10Go de RAM
              threads = "4")) # On plafonne l'utilise par duckdb de 4 coeurs (sur les 30 dispos sur Cerise)
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.21 Consulter le contenu d'une base

Reprenons la base `BDD_formation.db` créée précédemment.

```
# On crée une nouvelle connexion
con <- DBI::dbConnect(drv = duckdb::duckdb(),
                      dbdir = "data/BDD_formation.db")

# Affichage de la liste des tables
dbListTables(con)
# > [1] "resultat_interm"
```

Ajoutons-y une vue :

```
# Création de la vue "RA2020_duckdb"
con |> duckdb::duckdb_register(
  name = "RA2020_duckdb",
  df = RA2020)
```

```
# Affichage de la liste des tables
dbListTables(con)
# > [1] "RA2020_duckdb"   "resultat_interm"
```

Qu'en est-il vraiment ?

```
DBI::dbGetQuery(con,"SELECT * FROM INFORMATION_SCHEMA.TABLES")
# table_catalog table_schema      table_name table_type
1 BDD_formation         main resultat_interm BASE TABLE
2          temp         main   RA2020_duckdb       VIEW
```

=> Après déconnexion, seule la table "resultat_interm" sera conservée dans la BDD.

```
# On se déconnecte
DBI::dbDisconnect(con, shutdown = TRUE)
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.22 Requêter des données Parquet extérieures (data.gouv) avec Cerise et duckdb (1/5)

Un exemple de code pour requêter directement la base Parquet des bureaux de vote directement depuis Cerise !!!

Configuration générale :

```r
# Installation d'une version récente de duckdb
install.packages("duckdb", repos = "https://packagemanager.posit.co/cran/__linux__/centos8/latest")

library(duckdb) ; library(glue)
cnx <- dbConnect(duckdb())

# Vérification de la version du package
dbGetQuery(cnx, "select version()")

dbExecute(cnx, "
  SET http_proxy='http://rie.proxy.national.agri:8080';
  SET http_proxy_username='${AGRICOLL_U}';
  SET http_proxy_password='${AGRICOLL_P}'")

# Installation et chargement de l'extension nécessaire httpfs
dbExecute(cnx, "INSTALL httpfs")
dbExecute(cnx, "LOAD httpfs")
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.23 Requêter des données Parquet extérieures (data.gouv) avec Cerise et duckdb (2/5)

Requête :

```
dataset <- "https://static.data.gouv.fr/resources/bureaux-de-vote-et-adresses-de-leurs-electeurs/20230626-135723/table-adresses-reu.parquet"

dbGetQuery(cnx, glue("DESCRIBE FROM '{dataset}'"))

dbDisconnect(cnx, shutdown = TRUE)
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.24 Requêter des données Parquet extérieures (data.gouv) avec Cerise et duckdb (3/5)

Le résultat de la requête :

	column_name	column_type	null	key	default	extra
1	code_commune_ref	VARCHAR	YES	<NA>	<NA>	<NA>
2	reconstitution_code_commune	VARCHAR	YES	<NA>	<NA>	<NA>
3	id_brut_bv_reu	VARCHAR	YES	<NA>	<NA>	<NA>
4	id	VARCHAR	YES	<NA>	<NA>	<NA>
5	geo_adresse	VARCHAR	YES	<NA>	<NA>	<NA>
6	geo_type	VARCHAR	YES	<NA>	<NA>	<NA>
7	geo_score	DOUBLE	YES	<NA>	<NA>	<NA>
8	longitude	DOUBLE	YES	<NA>	<NA>	<NA>
9	latitude	DOUBLE	YES	<NA>	<NA>	<NA>
10	api_line	VARCHAR	YES	<NA>	<NA>	<NA>
11	nb_bv_commune	UINTEGER	YES	<NA>	<NA>	<NA>
12	nb_adresses	UINTEGER	YES	<NA>	<NA>	<NA>

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.25 Requêter des données Parquet extérieures (data.gouv) avec Cerise et duckdb (4/5)

Autre exemple pour requêter la table Parquet des unités légales de SIRENE :

Requête à passer après configuration du proxy et chargement de l'extension https :

```
cnx <- dbConnect(duckdb())

dataset <- "https://static.data.gouv.fr/resources/base-sirene-des-entreprises-et-de-leurs-etablissements-siren-siret/20231214-131836/stockuni

req <- glue_sql("
CREATE TABLE extrait_siren AS
FROM {dataset}
WHERE SIREN in ('345149587','388467441')
", .con = cnx)

dbSendQuery(cnx, req)

recup_siren <- dbReadTable(cnx,"extrait_siren")

dbDisconnect(cnx, shutdown = TRUE)
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.26 Requêter des données Parquet extérieures (data.gouv) avec Cerise et duckdb (4/4)

Le résultat de la requête :

```
# A tibble: 2 × 34
  siren     statutDiffusionUniteLegale unitePurgeeUniteLegale dateCreationUniteLeg…¹ sigleUniteLegale sexeUniteLegale prenom1UniteLegale pren
  <chr>     <chr>                      <lgl>                  <date>                 <chr>            <chr>           <chr>              <chr
1 345149587 O                          NA                     1985-01-01             INALCA           NA              NA                 NA
2 388467441 O                          NA                     1992-09-01             NA               NA              NA                 NA
# ℹ abbreviated name: ¹dateCreationUniteLegale
# ℹ 25 more variables: prenom4UniteLegale <chr>, prenomUsuelUniteLegale <chr>, pseudonymeUniteLegale <chr>, identifiantAssociationUniteLegale
#   trancheEffectifsUniteLegale <chr>, anneeEffectifsUniteLegale <dbl>, dateDernierTraitementUniteLegale <dttm>, nombrePeriodesUniteLegale <d
#   anneeCategorieEntreprise <dbl>, dateDebut <date>, etatAdministratifUniteLegale <chr>, nomUniteLegale <chr>, nomUsageUniteLegale <chr>, de
#   denominationUsuelle1UniteLegale <chr>, denominationUsuelle2UniteLegale <chr>, denominationUsuelle3UniteLegale <chr>, categorieJuridiqueUn
#   activitePrincipaleUniteLegale <chr>, nomenclatureActivitePrincipaleUniteLegale <chr>, nicSiegeUniteLegale <chr>, economieSocialeSolidaire
#   societeMissionUniteLegale <chr>, caractereEmployeurUniteLegale <chr>
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.27 Exercice 3

Exercice 3 (manipulation avec duckdb)

- Avec duckdb, se connecter au fichier du RA2020 au format Parquet (cf. exercice 1)
- Filtrer la table du RA2020 EXPLOITATIONS sur la région de votre choix (SIEGE_REG) puis calculer la moyenne de la SAU (SAU_TOT) en fonction de l'orientation nomenclature agrégée (OTEFDA_COEF17).
- Réaliser le même traitement que précédemment mais sur une région différente.
- Fusionner les 2 tables créées aux questions précédentes de manière à avoir une table qui se présente sous la forme suivante :

OTEFDA_COEF17	MOY_SAU_76	MOY_SAU_93
5074	65905.17	2586.15
3900	77613.40	53591.05
4800	573678.07	216192.42
9000	6308.55	554.31
4700	78358.02	8047.61
3500	315107.89	104019.71
6184	340337.60	54778.68
2829	17959.11	18192.31
4500	122406.67	7476.54
1516	900230.49	111232.37
4600	633306.48	51781.59

- Re-faites l'exercice avec SQL (ou la syntaxe dplyr)

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

4.28 Pour finir, une comparaison entre {arrow} et {duckdb} (1/2)

Tableau repris de la documentation **utilitr**.

Je souhaite…	arrow	duckdb
Optimiser mes traitements pour des données volumineuses	✔️	✔️
Travailler sur un fichier .parquet ou .csv sans le charger entièrement en mémoire	✔️	✔️
Utiliser la syntaxe `dplyr` pour traiter mes données	✔️	✔️
Utiliser du langage SQL pour traiter mes données	❌	✔️
Joindre des tables très volumineuses (plus de 4 Go)	❌	✔️
Utiliser des fonctions fenêtres	❌	✔️
Utiliser des fonctions statistiques qui n'existent pas dans arrow	❌	✔️
Écrire un fichier .parquet	✔️	✔️

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE
Liberté
Égalité
Fraternité

4.29 Pour finir, une comparaison entre {arrow} et {duckdb} (2/2)

{arrow} et {duckdb} partagent de nombreux concepts. Voici quelques différences :

- **{duckdb} comprend parfaitement SQL**. Si vous êtes à l'aise avec ce langage, vous ne serez pas dépaysés.
- Le projet duckdb est très récent. Il y a régulièrement des évolutions qui sont souvent des extensions ou des optimisations, et parfois la résolution de bugs. **arrow est un projet plus ancien et plus mature**.
- **La couverture fonctionnelle des fonctions standards de R est meilleure sur {duckdb} que sur {arrow}.**
  Il est préférable d'utiliser {duckdb} pour les jointures de tables volumineuses.
  De même, les fonctions `pivot_wider()`, `pivot_longer()` et les `windows_function` existent nativement dans duckdb mais pas dans arrow. Par exemple :

```r
# arrow ne peut pas exécuter ceci
bpe_ens_2022_dataset |>
  group_by(DEP) |>
  mutate(NB_EQUIP_TOTAL_DEP  = sum(NB_EQUIP)) |>
  select(DEP, NB_EQUIP, NB_EQUIP_TOTAL_DEP)
```

```
# Source:   SQL [?? x 3]
# Database: DuckDB v0.9.2 [unknown@Linux 6.5.0-1024-azure:R 4.3.2/:m
# Groups:   DEP
   DEP   NB_EQUIP NB_EQUIP_TOTAL_DEP
   <chr>    <dbl>              <dbl>
 1 09           2               7316
 2 09           3               7316
# ℹ more rows
```

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5 Pour en savoir plus

MINISTÈRE
DE L'AGRICULTURE
ET DE LA SOUVERAINETÉ
ALIMENTAIRE

Liberté
Égalité
Fraternité

5.1 Conseil de lecture

Pour ceux qui veulent aller plus loin :

- Comment bien utiliser le package `arrow` ? C'est par **ici**
- Comment utiliser DuckDB sur des fichiers au format Parquet ? C'est par **là**
- Comment requêter des fichiers au format Parquet avec Python ? Lien vers la **documentation**.
- Why Parquet Is the Go-To Format for Data Engineer. Article en anglais très pointu sur la constitution et le fonctionnement des fichiers au format Parquet. Un must-read à lire par **ici**.

Note interne écrite par le DEMESIS : voir **ici**